

Capítulo 2

Gramáticas de concatenación de rango y el SAT como lenguaje formal

Las gramáticas de concatenación de rango son el formalismo sobre el que se desarrolla el resto del trabajo. El capítulo se organiza en tres secciones. La sección 2.1 introduce las gramáticas de concatenación de rango, su presentación intuitiva, sus definiciones formales y un ejemplo de reconocimiento. La sección 2.2 expone las propiedades de clausura del formalismo y su caracterización en términos de complejidad. La sección 2.3 revisa los antecedentes directos del trabajo: los estudios previos de la Facultad de Matemática y Computación de la Universidad de La Habana que abordan el SAT mediante formalismos de la teoría de lenguajes.

2.1. Gramáticas de concatenación de rango

Las gramáticas de concatenación de rango (RCG) [4] son un formalismo propuesto en 1998 por Pierre Boullier, un investigador en el campo de la lingüística computacional. El objetivo inicial del formalismo era proporcionar un modelo más expresivo que las gramáticas libres del contexto para describir ciertos fenómenos del lenguaje natural, como los números chinos y el orden variable de algunas palabras en alemán [5].

Antes de entrar a la presentación del formalismo, es necesario precisar una convención terminológica que rige a lo largo de todo el documento. El formalismo que se presenta en este capítulo corresponde, en la literatura, a las **gramáticas de concatenación de rango positivas** (pRCG), una de las dos variantes del formalismo general de Boullier. La otra variante, las gramáticas de concatenación de rango con negación, admite cláusulas con predicados negados y no se utiliza en este trabajo. La restricción a la variante positiva es importante por-

que, como se establece más adelante, los lenguajes que se reconocen por una pRCG son exactamente aquellos cuyo problema de la palabra está en P , y los resultados de los capítulos posteriores se apoyan en dicha caracterización. Para abreviar, a lo largo del documento se adopta el nombre **gramática de concatenación de rango** (RCG) para referirse, salvo mención explícita, a la variante positiva.

2.1.1. Presentación intuitiva

A continuación se introducen las nociones de predicado, argumento, sustitución de rango y derivación, que se utilizan más adelante.

A los no terminales de una RCG se les llama **predicados**. Cada predicado tiene un conjunto de argumentos. A la cantidad de argumentos de un predicado se le denomina **aridad**. Por ejemplo, $A(X, Y)$ denota el predicado A con argumentos X e Y ; en este caso, la aridad de A es 2.

Cada argumento de un predicado puede estar formado por variables y terminales. Por convenio, las variables se denotan por letras mayúsculas del final del alfabeto, y los terminales con letras minúsculas. Con este convenio, el predicado $B(aX, XY, abZ)$ tiene aridad 3. El primer argumento está formado por el terminal a y la variable X . El segundo argumento está formado por la concatenación de las variables X e Y . El tercer argumento está formado por la concatenación de los terminales a y b y la variable Z .

Cada predicado reconoce un vector de cadenas cuya dimensión es la aridad del predicado. Cada cadena del vector se asocia a un argumento del predicado.

Por ejemplo, si al predicado $A(X, Y)$ se le asocia el vector $[abc, d]$, al primer argumento de A se le asigna la cadena abc y al segundo la cadena d . Esto significa que $X = abc$ y que $Y = d$.

De manera similar, si al predicado $B(aX, XY, abZ)$ se le asigna el vector $[aa, de, abcc]$, al primer argumento de B se le asigna la cadena aa , al segundo la cadena de y al tercero la cadena $abcc$. Es decir, $aX = aa$, $XY = de$ y $abZ = abcc$. Como X es una variable y se tiene que $aX = aa$, el único valor que puede tomar X es $X = a$. En el tercer argumento ocurre algo similar: como $abZ = abcc$, para que se cumpla la igualdad el único valor posible para Z es $Z = cc$. En el segundo argumento, las variables X e Y deben tomar valores tales que su concatenación sea de ; esto se puede hacer de tres formas: $X = d$, $Y = e$; $X = de$, $Y = \varepsilon$; y $X = \varepsilon$, $Y = de$.

Si al predicado $B(aX, XY, abZ)$ se le asigna el vector $[bb, de, abcc]$, entonces al tenerse $aX = bb$, X no puede tomar ningún valor: el primer carácter b de la cadena no coincide con el terminal a que encabeza el argumento aX . Por tanto, el predicado B no reconoce este vector.

A las producciones de esta gramática se les denomina **cláusulas**. El lado izquierdo de la producción siempre está formado por un único predicado. El lado derecho puede contener cualquier cantidad de predicados con sus respectivos argumentos, o la cadena vacía. Un ejemplo de cláusula es

$$A(XYZ, W) \rightarrow B(X) C(XY, Z) D(W).$$

La regla anterior tiene el siguiente significado: el predicado A recibe un vector de dimensión 2; a partir de las cadenas del vector, se asignan valores a las variables X , Y , Z y W , y con esos valores se construyen los vectores que reciben los predicados B , C y D .

El proceso se ilustra con un ejemplo en el que el predicado A recibe el vector de cadenas $[abc, d]$. El primer paso es asignar las cadenas del vector a los argumentos del predicado. El primer argumento de A es abc y el segundo es d . Como el primer argumento de A está definido como XYZ y el segundo como W , debe cumplirse que $XYZ = abc$ y $W = d$. Esto significa que las variables X , Y y Z deben tomar los valores posibles cuya concatenación sea abc , mientras que W solo puede tomar el valor d . En el cuadro 2.1 se muestran algunas asignaciones posibles para las variables X , Y , Z y W a partir del vector de entrada.

X	Y	Z	W
ε	ε	abc	d
ε	a	bc	d
ε	ab	c	d
ε	abc	ε	d
a	ε	bc	d
a	b	c	d
a	bc	ε	d
ab	ε	c	d
ab	c	ε	d
abc	ε	ε	d

Cuadro 2.1: Asignaciones posibles para las variables X , Y , Z y W a partir del vector $[abc, d]$.

Continuando con el ejemplo, se supone que los valores de X , Y y Z son $X = ab$, $Y = \varepsilon$ y $Z = c$. A partir de esta asignación se construyen los vectores con los que se evalúan los predicados del lado derecho $B(X) C(XY, Z) D(W)$. Como $X = ab$, $Y = \varepsilon$, $Z = c$ y $W = d$, el lado derecho se evalúa con los vectores $[ab]$, $[ab, c]$ y $[d]$, y el resultado es la derivación $B(ab) C(ab, c) D(d)$.

El proceso de asignar los vectores a los argumentos, identificar los valores de las variables e instanciar los lados derechos se repite en cada uno de los predicados del lado derecho de la cláusula.

Existe otro tipo de producciones, denominadas cláusulas terminales, en las que un predicado deriva en ε para determinados valores de sus argumentos. Por ejemplo,

$$B(ab) \rightarrow \varepsilon, \quad C(ab, c) \rightarrow \varepsilon, \quad D(d) \rightarrow \varepsilon.$$

La forma general de estas producciones es $A(x_1, \dots, x_k) \rightarrow \varepsilon$, donde k es la aridad de A y cada x_i es un argumento de A , esto es, una cadena de variables y terminales.

Un predicado reconoce un vector de cadenas si existe una asignación de sus argumentos a valores de las variables y una secuencia de derivaciones que

termina en la cadena vacía. Por ejemplo, en el caso de $B(ab) \rightarrow \varepsilon$, el predicado B reconoce la cadena ab .

A modo de ilustración, se considera el siguiente conjunto de producciones:

1. $A(XYZ, W) \rightarrow B(X) C(XY, Z) D(W)$,
2. $B(ab) \rightarrow \varepsilon$,
3. $C(ab, c) \rightarrow \varepsilon$,
4. $D(d) \rightarrow \varepsilon$.

Con estas producciones, el predicado A reconoce el vector $[abc, d]$: con la asignación $X = ab$, $Y = \varepsilon$, $Z = c$ y $W = d$, el predicado A deriva en $B(ab) C(ab, c) D(d)$, y cada uno de los predicados $B(ab)$, $C(ab, c)$ y $D(d)$ deriva en la cadena vacía.

Un predicado no reconoce un vector de cadenas cuando para ninguna asignación de variables se deriva en la cadena vacía.

Presentadas estas nociones, a continuación se introducen las definiciones formales de las gramáticas de concatenación de rango.

2.1.2. Definiciones formales

El formalismo toma su nombre de la noción de rango, que identifica las subcadenas asignadas a las variables.

Definición 2.1. Un **rango** de una cadena es un par (i, j) de posiciones, con i y j enteros no negativos tales que $i \leq j$, que delimita la subcadena comprendida entre la posición i y la posición j .

Por ejemplo, con las posiciones indexadas desde 0, para la cadena $abcd$ el rango $(1, 3)$ delimita la subcadena bc .

Definición 2.2. Una **gramática de concatenación de rango** se define como una 5-tupla

$$G = (N, T, V, P, S),$$

donde:

- N es un conjunto finito de **predicados** o símbolos no terminales; cada predicado tiene una aridad, que indica la dimensión del vector de cadenas que reconoce;
- T es un conjunto finito de **símbolos terminales**;
- V es un conjunto finito de **variables**;
- P es un conjunto finito de **cláusulas** de la forma

$$A(x_1, x_2, \dots, x_k) \rightarrow B_1(y_{1,1}, y_{1,2}, \dots, y_{1,m_1}) \dots B_n(y_{n,1}, y_{n,2}, \dots, y_{n,m_n}),$$

donde $A, B_i \in N$, $x_i, y_{i,j} \in (V \cup T)^*$, $n \geq 0$, k es la aridad de A y m_i es la aridad de B_i ; cuando $n = 0$, el lado derecho es la cadena vacía y la cláusula se denomina terminal;

- $S \in N$ es el **predicado inicial** de la gramática, que siempre tiene aridad 1.

Por ejemplo, a continuación se muestra una gramática de concatenación de rango que reconoce el lenguaje $L_{\text{copy}}^3 = \{w^3 \mid w \in \{a, b, c\}^*\}$:

$$G_{\text{copy}}^3 = (N, T, V, P, S),$$

donde $N = \{A, S\}$, $T = \{a, b, c\}$, $V = \{X, Y, Z\}$, el símbolo inicial es S y el conjunto de cláusulas P es el siguiente:

1. $S(XYZ) \rightarrow A(X, Y, Z)$,
2. $A(aX, aY, aZ) \rightarrow A(X, Y, Z)$,
3. $A(bX, bY, bZ) \rightarrow A(X, Y, Z)$,
4. $A(cX, cY, cZ) \rightarrow A(X, Y, Z)$,
5. $A(\varepsilon, \varepsilon, \varepsilon) \rightarrow \varepsilon$.

A diferencia de las gramáticas presentadas en el capítulo anterior, que se definen a partir de un mecanismo de generación de cadenas, las RCG se formulan directamente como un mecanismo de reconocimiento. Su funcionamiento consiste en determinar si una cadena pertenece o no al lenguaje.

Definición 2.3. Una **sustitución de rango** es un mecanismo que asigna a cada variable un rango de la cadena que se reconoce, de acuerdo con la estructura del argumento. La variable se reemplaza entonces por la subcadena que ese rango delimita.

Por ejemplo, dado el predicado $A(Xa)$ con $X \in V$ y $a \in T$, la estructura del argumento de A es una variable X seguida del terminal a . Si el predicado A recibe la cadena baa , entonces X se puede asociar con la subcadena ba de la cadena original: si $X = ba$, entonces $Xa = baa$. La variable X no puede tomar el valor baa , porque ningún carácter de la cadena de entrada coincide con el terminal a al final del argumento. La variable X tampoco puede tomar el valor b , porque con ese valor el argumento Xa no cubre la cadena completa.

Definición 2.4. El **tamaño** de una RCG $G = (N, T, V, P, S)$, que se denota $|G|$, es

$$|G| = |N| + |T| + |P|,$$

la suma de las cantidades de predicados, símbolos terminales y cláusulas.

Definición 2.5. Las **gramáticas de concatenación de rango simple** (sRCG) son un subconjunto de las RCG que restringen la forma de las cláusulas. Una RCG G es **simple** si los argumentos del lado derecho de cada cláusula son variables distintas, y todas estas variables aparecen una sola vez en los argumentos del lado izquierdo.

Por ejemplo, la cláusula $A(XY) \rightarrow B(X)C(Y)$ cumple la condición de simplicidad: los argumentos del lado derecho, X e Y , son variables distintas, y cada una aparece una sola vez en el lado izquierdo. La cláusula $A(X) \rightarrow B(X)C(X)$ no la cumple, porque la variable X aparece en dos argumentos del lado derecho; una RCG que la contenga no es simple.

Las sRCG son equivalentes a los sistemas lineales de reescritura libres del contexto (LCFRS) introducidos en [9]. Este subconjunto se utiliza en [1] para una variante del SAT denominada **Satisfacibilidad Booleana de Concatenación de Rango Simple**, que se discute en la sección 2.3.

Definición 2.6. Una RCG es **no lineal** si una misma variable aparece más de una vez en el lado derecho de una cláusula.

Por ejemplo, la cláusula $A(X) \rightarrow B(X)C(X)$ es no lineal, pues la variable X aparece en dos argumentos del lado derecho.

Una vez introducidas las definiciones formales, a continuación se presenta un ejemplo completo de reconocimiento de una cadena por una RCG, aplicando los mecanismos descritos en esta sección.

2.1.3. Ejemplo de reconocimiento

La presentación intuitiva describió cómo se asocian los elementos de un vector a los argumentos de un predicado y cómo se construyen los vectores que reciben los predicados del lado derecho de una cláusula. En esta subsección se aplica ese proceso a un caso completo: el reconocimiento de la cadena $abcabcabc$ por la gramática G_{copy}^3 presentada anteriormente.

La cadena $abcabcabc$ se reconoce por G_{copy}^3 mediante la siguiente secuencia de derivaciones:

$$S(abcabcabc) \rightarrow A(abc, abc, abc) \rightarrow A(bc, bc, bc) \rightarrow A(c, c, c) \rightarrow A(\varepsilon, \varepsilon, \varepsilon) \rightarrow \varepsilon.$$

A continuación se describen los pasos.

- **Primer paso:** en la primera cláusula $S(XYZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = abc, Y = abc$ y $Z = abc$. De esta forma se deriva en el predicado $A(abc, abc, abc)$.
- **Segundo paso:** en la segunda cláusula $A(aX, aY, aZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = bc, Y = bc$ y $Z = bc$. Con estos valores se deriva en el predicado $A(bc, bc, bc)$.
- **Tercer paso:** en la tercera cláusula $A(bX, bY, bZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = c, Y = c$ y $Z = c$. Con estos valores se deriva en el predicado $A(c, c, c)$.
- **Cuarto paso:** en la cuarta cláusula $A(cX, cY, cZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = \varepsilon, Y = \varepsilon$ y $Z = \varepsilon$. Con estos valores se deriva en el predicado $A(\varepsilon, \varepsilon, \varepsilon)$.

- **Quinto paso:** en la última cláusula $A(\varepsilon, \varepsilon, \varepsilon) \rightarrow \varepsilon$ se deriva en la cadena vacía, con lo que la cadena $abcbabc$ queda reconocida.

Una vez presentado el proceso de derivación, en la siguiente sección se analizan las propiedades del formalismo: clausura bajo intersección, comparación con las gramáticas libres del contexto y complejidad del problema de la palabra.

2.2. Propiedades de las RCG

Las RCG son cerradas bajo unión, intersección y complemento [4]. De estas tres operaciones, la clausura bajo intersección es la que importa para este trabajo. En el enfoque que se revisa en la sección 2.3, la satisfacibilidad de una fórmula se decide según si una intersección de dos lenguajes es vacía. En esta sección se presentan estas propiedades de clausura y la caracterización del reconocimiento en términos de complejidad. La clausura bajo intersección se usa en el capítulo ??, donde se construye una RCG cuyo lenguaje es esa intersección. El reconocimiento polinomial se usa en el capítulo ??, donde se caracteriza la decisión de su vacío.

Teorema 2.1 (Clausura bajo intersección). *Si L_1 y L_2 son lenguajes reconocidos por RCG, entonces $L_1 \cap L_2$ también lo es [4].*

Las RCG son más expresivas que las gramáticas libres del contexto. Por ejemplo, para todo $k \geq 2$, el lenguaje $L_{\text{copy}}^k = \{w^k \mid w \in \Sigma^*\}$ es dependiente del contexto y no libre del contexto [8]. Aun así, se reconoce por una RCG. En particular, el lenguaje L_{copy}^3 se reconoce por la gramática G_{copy}^3 del ejemplo anterior.

La próxima subsección analiza el problema de la palabra de las RCG y su complejidad.

2.2.1. Complejidad y caracterización en términos de P

El siguiente resultado es una caracterización del reconocimiento de las RCG en términos de complejidad.

Teorema 2.2 (Caracterización en términos de P). *Un lenguaje se reconoce por una RCG en su variante positiva si y solo si su problema de la palabra está en P [3, 6].*

La dirección que afirma que todo lenguaje que se reconoce por una RCG tiene su problema de la palabra en P se sigue de la existencia de un algoritmo de reconocimiento polinomial para el formalismo, que se describe a continuación.

El algoritmo de reconocimiento estándar para las RCG utiliza un proceso de memorización sobre las cadenas asignadas a los argumentos de los predicados [4]. La cantidad máxima de rangos de una cadena de longitud n es $O(n^2)$, y la máxima aridad de un predicado es constante, por lo que el proceso de memorización emplea una cantidad polinomial de estados. La complejidad del

algoritmo es $O(|P|n^{2h(l+1)})$, donde h es la máxima aridad de un predicado, l es la máxima cantidad de predicados en el lado derecho de una cláusula y n es la longitud de la cadena que se reconoce. El teorema establece, además, la dirección inversa: todo lenguaje cuyo problema de la palabra está en P se reconoce por una RCG.

El alcance exacto de la caracterización y sus consecuencias para la relación entre formalismos gramaticales y el SAT son el objeto central de los capítulos posteriores.

Una vez establecida la caracterización de las RCG como formalismo que reconoce exactamente los lenguajes cuyo problema de la palabra está en P , en la siguiente sección se revisan los antecedentes del trabajo.

2.3. Antecedentes en el estudio del SAT mediante formalismos de la teoría de lenguajes

El estudio del problema de la satisfacibilidad booleana mediante formalismos de la teoría de lenguajes cuenta con antecedentes en la Facultad de Matemática y Computación de la Universidad de La Habana. El presente trabajo se apoya en dos de ellos: el problema de la satisfacibilidad booleana libre del contexto [7], abreviado SATCF, y el de la satisfacibilidad booleana de concatenación de rango simple [1], abreviado SATSRC.

SATCF y SATSRC comparten una misma estrategia. A una fórmula booleana se le asocia un lenguaje regular que representa sus interpretaciones verdaderas, bajo el supuesto de que cada ocurrencia de variable booleana se trata por separado. Sobre ese lenguaje se impone después que las ocurrencias de una misma variable booleana tomen el mismo valor, mediante una gramática. La satisfacibilidad se decide según si la intersección de ambos lenguajes es vacía.

La sección se organiza según esa estrategia. En la subsección 2.3.1 se describe el autómata con que se construye el lenguaje regular de las interpretaciones. En la subsección 2.3.2 se presentan la gramática que impone la consistencia, la clase de fórmulas que cada trabajo resuelve y la razón por la que las RCG son el siguiente formalismo de esta línea de trabajo.

2.3.1. El autómata de las interpretaciones

El lenguaje regular de las interpretaciones se construye mediante un autómata finito, el **autómata booleano** [7]. Su alfabeto de entrada es $\{0, 1\}$, cuyos símbolos son los valores de verdad: 1 el verdadero y 0 el falso. Cada ocurrencia de variable booleana de la fórmula aporta uno de esos símbolos, según su valor. Una cadena de ceros y unos describe así una asignación a las ocurrencias, tratadas por separado, y se acepta cuando esa asignación hace verdadera la fórmula.

El autómata de una sola ocurrencia consta de un estado inicial q_0 y dos estados más, V y F . Al leer 1 se pasa a V y al leer 0 se pasa a F . El estado

V es de aceptación y representa que la fórmula resulta verdadera; el estado F representa lo contrario. La figura 2.1 lo muestra.

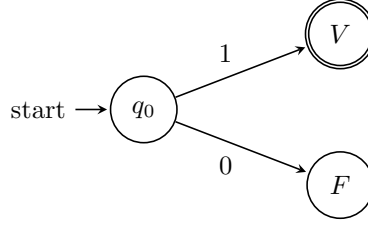


Figura 2.1: Autómata booleano de una ocurrencia de variable.

Los autómatas de las ocurrencias se combinan según los operadores de la fórmula [7]. El autómata de una fórmula compuesta se obtiene de los autómatas de sus subfórmulas, según el operador principal.

La negación se aplica a una subfórmula. Sea A una subfórmula y B su autómata booleano; con la negación se intercambian el estado de aceptación V y el de rechazo F , de modo que el autómata de $\neg A$ es B con V como no aceptante y F como aceptante. Cada cadena que hace verdadera a A hace falsa a $\neg A$, y al revés. La figura 2.2 lo muestra.

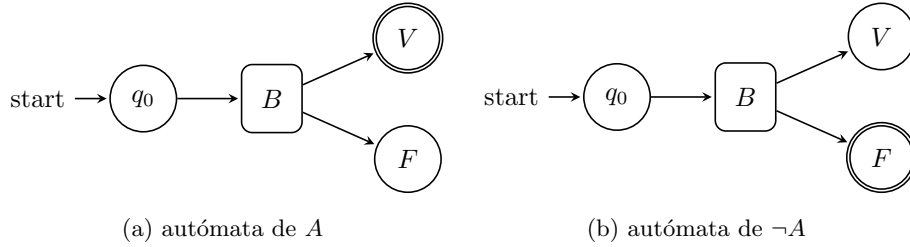


Figura 2.2: La negación intercambia el papel de V y F : en $\neg A$ el estado aceptante es F .

La conjunción y la disyunción se aplican a dos subfórmulas. Sean A_1 y A_2 dos subfórmulas y B_1 , B_2 sus autómatas booleanos, con estados de aceptación V_1 , V_2 y de rechazo F_1 , F_2 .

En la disyunción $A_1 \vee A_2$ se concatenan los autómatas de A_1 y de A_2 . Si en el autómata de A_1 se alcanza el estado V_1 , la disyunción ya resulta verdadera; desde ahí, a lo largo de un **camino de extensión**¹, se consumen los símbolos de A_2 hasta el estado V del autómata combinado. Si se alcanza el estado F_1 , el valor depende de A_2 , y por eso el estado F_1 se une con el estado inicial del autómata de A_2 ; los estados V_2 y F_2 pasan a ser los del autómata combinado. La figura 2.3 lo muestra.

¹Una sucesión de estados en la que se consumen los símbolos restantes de la cadena y se llega al estado ya determinado, cualquiera que sea su valor.

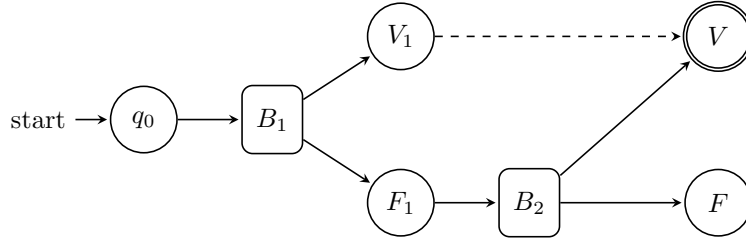


Figura 2.3: Disyunción $A_1 \vee A_2$. Al alcanzar V_1 , un camino de extensión (flecha discontinua) lleva al estado aceptante V ; al alcanzar F_1 , el cómputo continúa en B_2 .

La conjunción $A_1 \wedge A_2$ se construye de forma análoga a la disyunción, con los papeles de V y F intercambiados. Si en el autómata de A_1 se alcanza el estado F_1 , la conjunción ya resulta falsa, y a lo largo de un camino de extensión se consumen los símbolos de A_2 hasta el estado F del autómata combinado. Si se alcanza el estado V_1 , el valor depende de A_2 , y el estado V_1 se une con el estado inicial del autómata de A_2 . La figura 2.4 lo muestra.

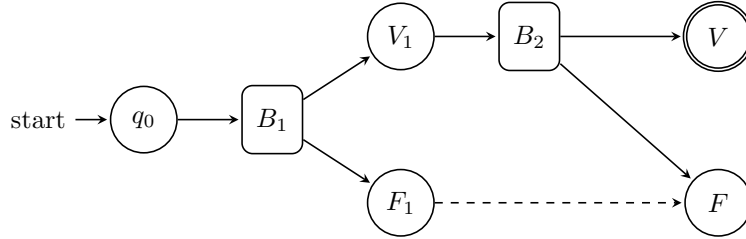


Figura 2.4: Conjunción $A_1 \wedge A_2$. Al alcanzar F_1 , un camino de extensión (flecha discontinua) lleva al estado F ; al alcanzar V_1 , el cómputo continúa en B_2 .

Por ejemplo, para la fórmula $A = x_1 \wedge (x_2 \vee x_1)$, cuya secuencia de ocurrencias es $x_1 x_2 x_1$, se construye primero el autómata de cada subfórmula, x_1 y $x_2 \vee x_1$. El de x_1 es el de una sola ocurrencia; el de $x_2 \vee x_1$ se obtiene por la regla de la disyunción. La figura 2.5 muestra ambos.

Al combinar los dos por la regla de la conjunción se obtiene el autómata de A : desde la rama verdadera de x_1 se entra en el autómata de $x_2 \vee x_1$, y desde la rama falsa se llega al estado F por un camino de extensión. La figura 2.6 lo muestra.

Un camino del estado inicial a V corresponde a una asignación que hace verdadera la fórmula cuando cada ocurrencia se trata por separado. Por ejemplo, con la cadena 110 se llega del estado inicial a V ; en ella se asigna 1 a la primera ocurrencia de x_1 , 1 a x_2 y 0 a la segunda ocurrencia de x_1 , valores con los que $x_1 \wedge (x_2 \vee x_1)$ resulta verdadera. En esa misma cadena, las dos ocurrencias de x_1 no toman el mismo valor, algo que no ocurre en ninguna interpretación. El

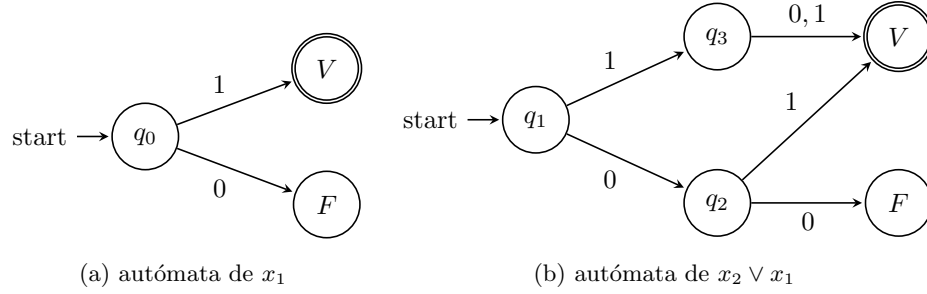


Figura 2.5: Autómatas de x_1 y de $x_2 \vee x_1$, las dos subfórmulas de A .

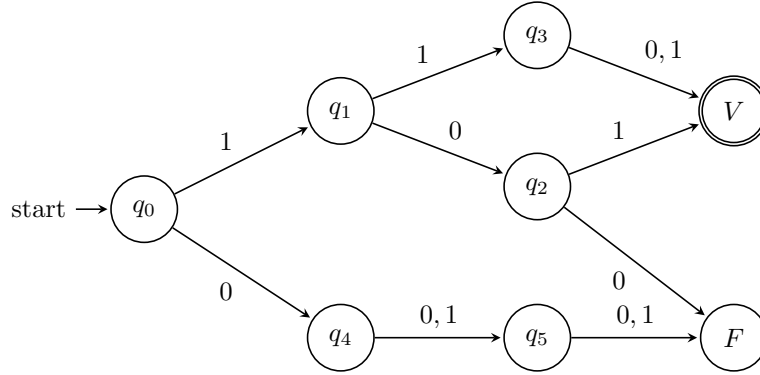


Figura 2.6: Autómata booleano de la fórmula $A = x_1 \wedge (x_2 \vee x_1)$. El estado V acepta las cadenas que hacen verdadera la fórmula cuando cada ocurrencia se trata por separado.

autómata booleano de una fórmula con n ocurrencias de variable booleana tiene $O(n^2)$ estados [7]. En él todas las ocurrencias se tratan por separado, incluso las de una misma variable booleana, y por eso hace falta un segundo mecanismo que imponga la consistencia entre las ocurrencias.

La próxima subsección presenta la gramática que impone esa consistencia y la clase de fórmulas que cada trabajo resuelve con ella.

2.3.2. La consistencia entre ocurrencias y la clase de fórmulas

Una cadena de ceros y unos, con un símbolo por ocurrencia, es **consistente** si en las posiciones de cada variable booleana lleva siempre el mismo valor. La gramática genera las cadenas consistentes de la longitud dada, y la intersección de ese lenguaje con el del autómata booleano contiene las interpretaciones verdaderas de la fórmula. La satisfacibilidad equivale entonces a que esa intersección no sea vacía. SATCF y SATSRC se diferencian en el formalismo de esa

gramática, y con ello en la clase de fórmulas que resuelven.

En SATCF la consistencia se impone con una gramática libre del contexto, equivalente a un autómata de pila [7]. En la pila del autómata se guarda el valor de una variable booleana en su primera ocurrencia y se compara en las siguientes. Como en una pila se accede a los valores en orden inverso al de guardado, con este esquema la consistencia solo se verifica cuando las ocurrencias están bien anidadas: si entre dos ocurrencias consecutivas de una variable booleana aparece una ocurrencia de otra, todas las ocurrencias de esa otra deben caer entre las dos primeras. Una secuencia de ocurrencias con esa propiedad tiene un **orden libre del contexto** [7]. Por ejemplo, en la secuencia $x_1 x_2 x_2 x_1$ las dos ocurrencias de x_2 caen entre las de x_1 ; la figura 2.7(a) muestra ese anidamiento. Con SATCF se resuelven en tiempo $O(n^3)$ las fórmulas cuyo orden es de ese tipo.

En SATSRC la consistencia se impone con una gramática de concatenación de rango simple [1]. Frente a la pila, que sigue un solo fragmento de la cadena, una sRCG admite varios a la vez, por lo que admite cruces entre ocurrencias que el orden libre del contexto excluye. Una secuencia para la que existe una sRCG de tamaño polinomial que genera sus cadenas consistentes tiene un **orden de concatenación de rango simple** [1]. Toda secuencia con orden libre del contexto lo tiene, y además lo tienen secuencias con cruces. Por ejemplo, en la secuencia $x_2 x_1 x_3 x_2 x_1$ las ocurrencias de x_2 y las de x_1 se cruzan, de modo que ese orden no es libre del contexto pero sí de concatenación de rango simple. La figura 2.7(b) muestra ese cruce. Con SATSRC se resuelven esas fórmulas en tiempo $O(n^k)$, donde k es la aridad de la sRCG, así que el grado del polinomio crece con la aridad necesaria para el cruce.

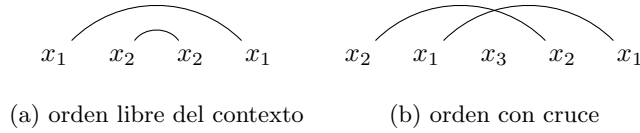


Figura 2.7: Arcos que unen las ocurrencias de cada variable booleana. En (a) los arcos se anidan; en (b), los de x_2 y los de x_1 se cruzan.

Una parte de las fórmulas ya se resuelve en tiempo polinomial. Las que no tienen cruces entre ocurrencias tienen orden libre del contexto y se deciden con SATCF; al aparecer cruces, la decisión sigue siendo polinomial con SATSRC mientras la aridad necesaria esté acotada. El problema empieza cuando no lo está: el exponente de la cota $O(n^k)$ crece con la cantidad de ocurrencias y la cota deja de ser polinomial.

Para codificar la consistencia de esas fórmulas hace falta la no linealidad. En una sRCG cada variable aparece una sola vez, de modo que, para igualar dos ocurrencias, hay que seguirlas en argumentos separados de la cláusula, y la cantidad de argumentos, esto es, la aridad, crece con el cruce. Con la no linealidad esa igualdad se expresa de forma directa: una misma variable aparece más de una vez en el lado derecho de una cláusula, y las subcadenas en que aparece deben coincidir, sin que la aridad tenga que crecer.

La no linealidad es justo la diferencia entre una sRCG y una RCG. Sin ella se reconocen los mismos lenguajes que con una sRCG, así que no se obtiene ninguna fórmula nueva. Por eso la siguiente clase natural de esta línea son las RCG.²

La clase es cerrada bajo intersección y el reconocimiento de la consistencia con una RCG sigue siendo polinomial, según los teoremas 2.1 y 2.2; pero el SAT no se decide por el reconocimiento, sino por el vacío de la intersección. En SATCF y SATSRC esa decisión es polinomial porque el vacío de las CFG y las sRCG lo es, no por una estrategia particular. Con la no linealidad de las RCG la aridez ya no crece con el cruce, pero el vacío de la intersección deja de ser polinomial, y ahí queda concentrada la dificultad.

En este capítulo se presentaron las gramáticas de concatenación de rango, con su reconocimiento polinomial y su clausura bajo intersección, y el esquema por intersección con que se decide la satisfacibilidad en SATCF y SATSRC. En el próximo capítulo se estudia ese vacío: se construye un procedimiento explícito de decisión y se caracteriza su costo.

²Entre las sRCG y las RCG está pMCFG, con una forma restringida de no linealidad. Este trabajo no la aborda: ya existe un trabajo de la Facultad en esa dirección [2].

Bibliografía

- [1] Manuel Aguilera López. Problema de la satisfacibilidad booleana de concatenación de rango simple. In *Jornada Científica Estudiantil MatCom 2016*, pages 1–5, La Habana, Cuba, 2016. Facultad de Matemática y Computación, Universidad de La Habana.
- [2] Jossué Arteché Muñoz. El problema de la satisfacibilidad booleana y el vacío de la intersección con lenguajes regulares. Trabajo de diploma, Universidad de La Habana, Facultad de Matemática y Computación, 5 2026.
- [3] Eberhard Bertsch and Mark-Jan Nederhof. On the complexity of some extensions of RCG parsing. In *Proceedings of the Seventh International Workshop on Parsing Technologies (IWPT 2001)*, pages 66–77, Beijing, China, 2001.
- [4] Pierre Boullier. Proposal for a natural language processing syntactic backbone. Technical Report RR-3342, INRIA-Rocquencourt, France, 1998.
- [5] Pierre Boullier. Chinese numbers, MIX, scrambling, and range concatenation grammars. In *Proceedings of the 9th Conference of the European Chapter of the Association for Computational Linguistics (EACL '99)*, pages 53–60, Bergen, Norway, 1999.
- [6] Pierre Boullier. Range concatenation grammars. In *Proceedings of the Sixth International Workshop on Parsing Technologies (IWPT 2000)*, pages 53–64, Trento, Italy, 2000. Association for Computational Linguistics.
- [7] Alina Fernández Arias. El problema de la satisfacibilidad booleana libre del contexto. un algoritmo polinomial. Tesis de licenciatura, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, 2007.
- [8] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson Addison-Wesley, Boston, MA, 3 edition, 2006.
- [9] K. Vijay-Shanker, David J. Weir, and Aravind K. Joshi. Characterizing structural descriptions produced by various grammatical formalisms. In

25th Annual Meeting of the Association for Computational Linguistics, pages 104–111, Stanford, California, USA, 1987. Association for Computational Linguistics.